Universidad Nacional de Loja
Carrera en Computación
Facultad de Energía las Industrias y los Recursos Naturales no Renovables

Componente Autónomo-AA

Diagramación de punteros y estados de memoria frente a la instanciación de objetos

Integrantes del Grupo 9	Isaac Vire, Danny Beltran, Axel Jimenez, Noelia Bustan, Lucas Orellana.		
Paralelo	A		
Professor	Ing. Cristian Narváez G. Mg.Sc.		
Unidad	1		
Fecha	18/10/2026	**Uso de IA:** Si	**Calificación:**

Introducción

La gestión de la memoria en JavaScript la realiza el Garbage Collector, que libera los objetos que ya no son alcanzables desde una raíz, como una variable global o el Stack. Cuando un objeto sigue siendo alcanzable aunque el programa ya no lo necesite, aparece una fuga de memoria, y por eso diagramar el Stack, el ámbito global y el Heap ayuda a seguir qué referencia lo mantiene vivo.

En este informe se analizan tres casos de estudio. En el Caso 1, Shared Lexical Context Leak, dos closure comparten un mismo Context y una cadena de referencias impide liberar arreglos de gran tamaño. En el Caso 2, Grafo Circular y Referencias Débiles, se analiza qué ocurre con un ciclo de nodos al eliminar su única referencia externa. En el Caso 3, Event Loop Queue y Retención por Callbacks, un emisor global conserva listeners que mantienen vivas sesiones ya cerradas. Para cada caso se presentan el código, los diagramas de los estados de la memoria y su explicación, y al final se plantean las conclusiones obtenidas.

Caso de Estudio 1: Shared Lexical Context Leak

1.1 Código a Diagramar:

```
let ejecutorGlobal = null;

function crearProcesadorResiduos() {
  // Asignación de 10 MB en el Heap
  let datosMasivos = new Array(10 * 1024 * 1024).fill("DATA_CRITICA");
  let copiaAnterior = ejecutorGlobal;

  // Función interna 1: NUNCA se invoca directamente, pero utiliza datosMasivos
  let funcionRetenida = function () {
    if (datosMasivos) {
      console.log("Referencia activa a datos masivos");
    }
  };

  // Función interna 2: Sobrevive asignándose a la variable global
  ejecutorGlobal = function () {
    console.log("Ejecutando proceso liviano...");
    if (copiaAnterior) {
      copiaAnterior();
    }
```

```
  };
}

// Simulación de llamadas repetidas
setInterval(() => {
  crearProcesadorResiduos();
  console.log(`Heap usado: ${(process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2)} MB`);
}, 1000);
```

1.2 Descripción del caso:

Se tiene una variable global llamada ejecutorGlobal, que empieza valiendo null, y una función llamada crearProcesadorResiduos, que se ejecuta cada segundo mediante setInterval. Cada vez que se ejecuta, la función crea un Array llamado datosMasivos de 10 485 760 elementos. Después guarda en copiaAnterior el valor que tenía ejecutorGlobal en ese momento. Luego crea dos funciones internas, funcionRetenida, que usa datosMasivos pero nunca se invoca, y una nueva ejecutorGlobal, que usa copiaAnterior y se asigna a la variable global, por lo que sobrevive cuando la función termina. Aunque ejecutorGlobal nunca use datosMasivos, ambas funciones son closure creadas en la misma ejecución, así que comparten un mismo Context. Esto, sumado a que copiaAnterior encadena cada ejecutorGlobal que se genere, provoca que el Garbage Collector no pueda liberar ningún Array y el Heap crezca sin parar hasta quedarse sin memoria.

1.3 Diagrama realizado con herramientas vectoriales:

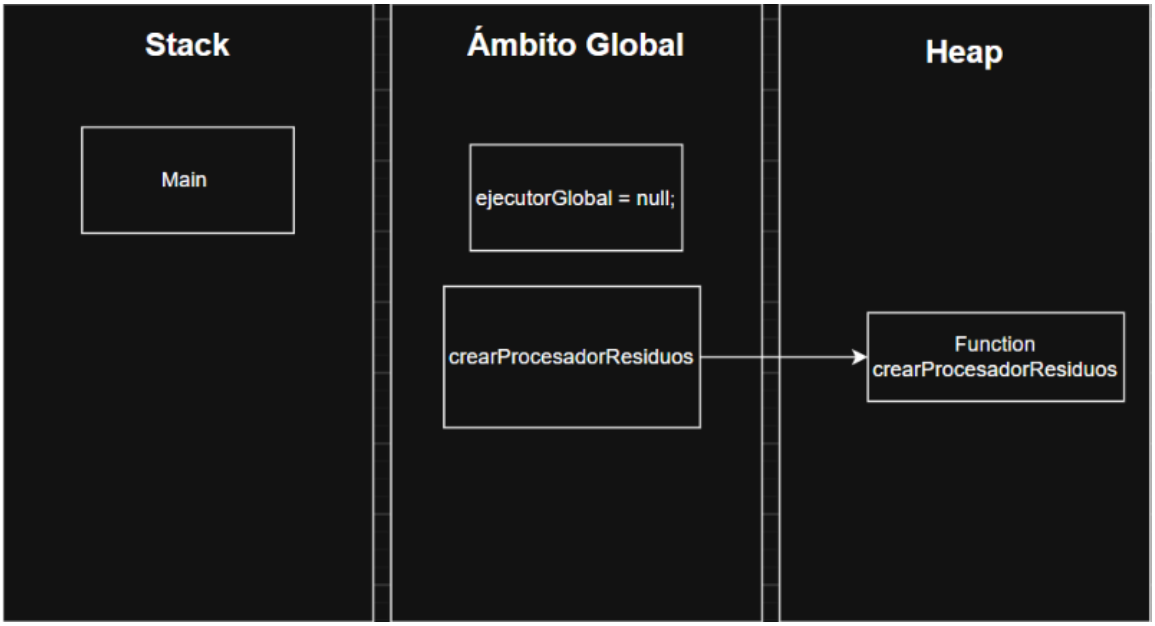


Figura 1: Fase 1

Fase 1: En el Stack aparece el main. En el ámbito global están ejecutorGlobal, que vale null, y crearProcesadorResiduos. Por último, en el Heap se guarda el objeto función crearProcesadorResiduos, que todavía no se ha ejecutado.

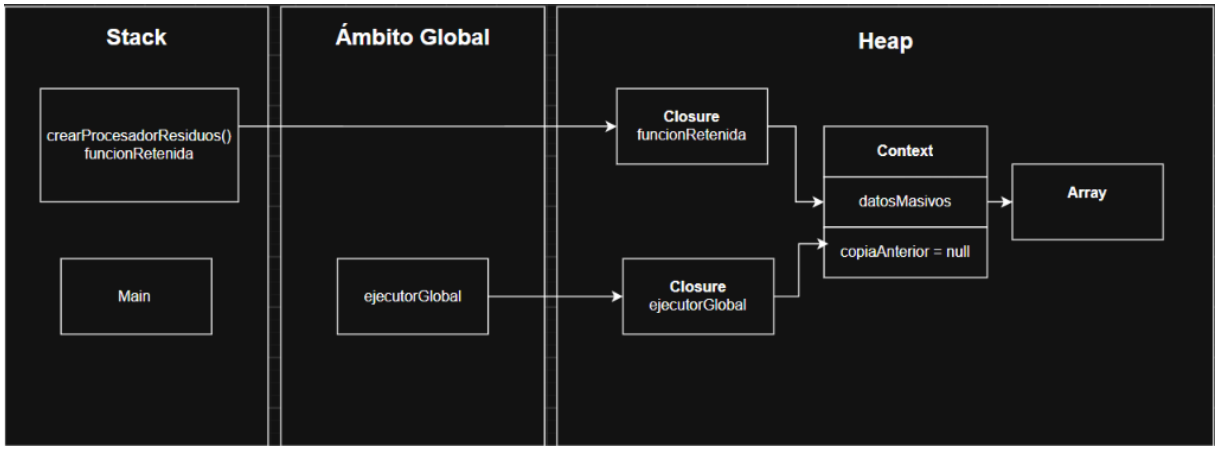


Figura 2: Fase 2

Fase 2: En el Stack se ejecuta crearProcesadorResiduos. Dentro de ella se crean dos closure, que no son más que funciones que conservan acceso a las variables del lugar donde fueron creadas. Estas son funcionRetenida, que solo se crea y nunca se ejecuta, y ejecutorGlobal, que al tomar otro valor dentro de crearProcesadorResiduos también se guarda en el Heap. Las variables que usan estas closure, datosMasivos y copiaAnterior, se guardan en un Context en el Heap, lo importante es que ambas closure apuntan al mismo Context. Por último, datosMasivos, dentro del Context, apunta a un Array con cientos de MB

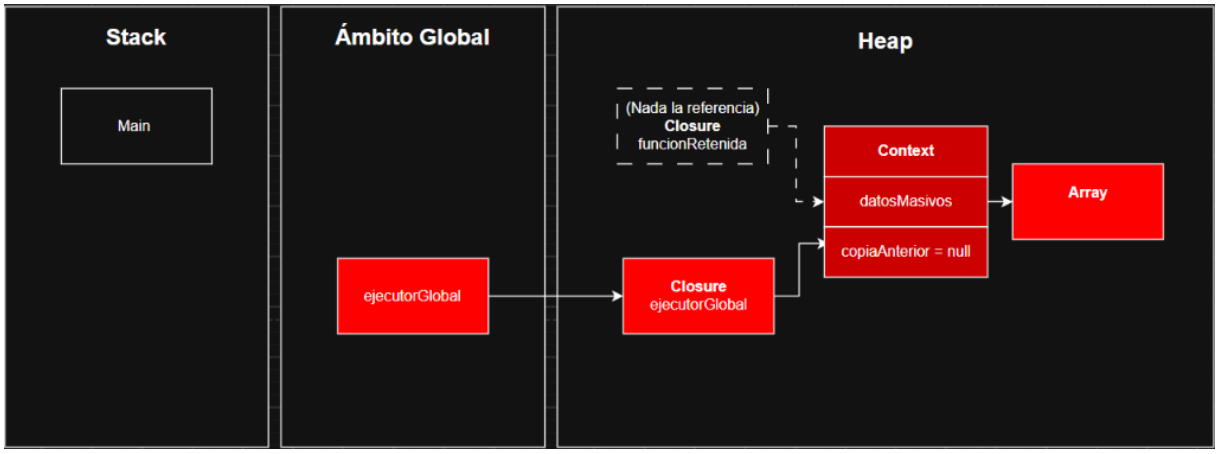


Figura 3: Fase 3

Fase 3: La función crearProcesadorResiduos cumple su ciclo y se deshecha del Stack, junto a funcionRetenida, que queda sin referencias y se convierte en basura. Solo queda ejecutorGlobal, que está referenciada por la variable global, y esta apunta a su Context. Aquí viene lo importante, los elementos que están rojos lo están porque se retienen y no se desechan. ¿Por qué? Porque los closure se almacenan en un mismo Context, lo que hace que, aunque uno de los closure desaparezca, el que queda siga permitiendo que se retenga lo que hay en él. El Garbage Collector no puede desechar el Array, porque es alcanzable por la cadena variable global → ejecutorGlobal → Context → datosMasivos → Array, aunque ejecutorGlobal nunca use datosMasivos.

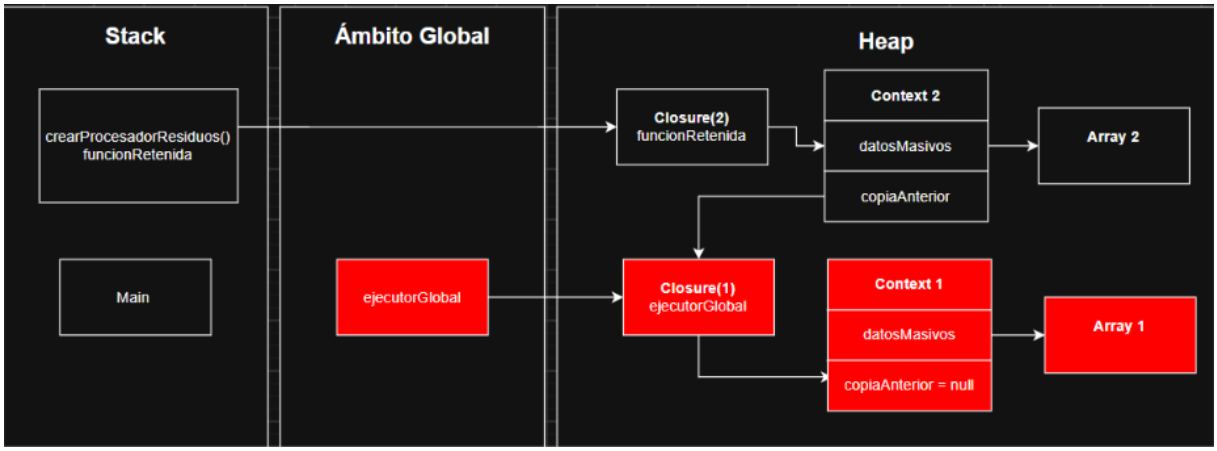


Figura 4: Fase 4

Fase 4: Para este momento crearProcesadorResiduos se vuelve a ejecutar, y crea otra funcionRetenida, otro Context y otro Array. Pero se sigue reteniendo el primer Context, la primera ejecutorGlobal y el primer Array. La razón es que se puede llegar a ellos gracias a copiaAnterior = ejecutorGlobal, que hace que el nuevo Context apunte a la primer ejecutorGlobal y, por ende, al resto de los elementos.

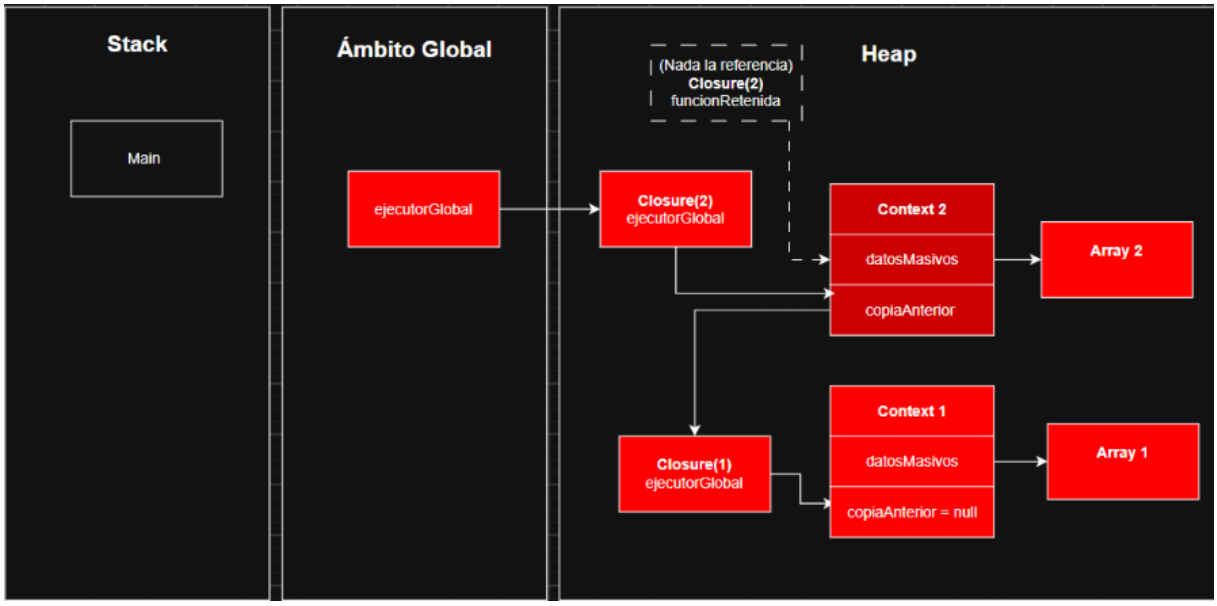


Figura 5: Fase 5

Fase 5: Esto también explica por qué, aunque ejecutorGlobal deje de referenciar a su primera ubicación en el Heap, el primer ejecutorGlobal no desaparece, pues sigue siendo alcanzable desde el segundo Context por medio de copiaAnterior. De aquí en más se forma una cascada, y cada vez que se ejecuta crearProcesadorResiduos se agrega un nuevo eslabón con su propio Array de cientos de MB. Esto hace que el Heap crezca de forma continua hasta quedarse sin memoria, porque todo lo de la ejecución anterior se queda en el Heap y nunca desaparece, aunque no se use.

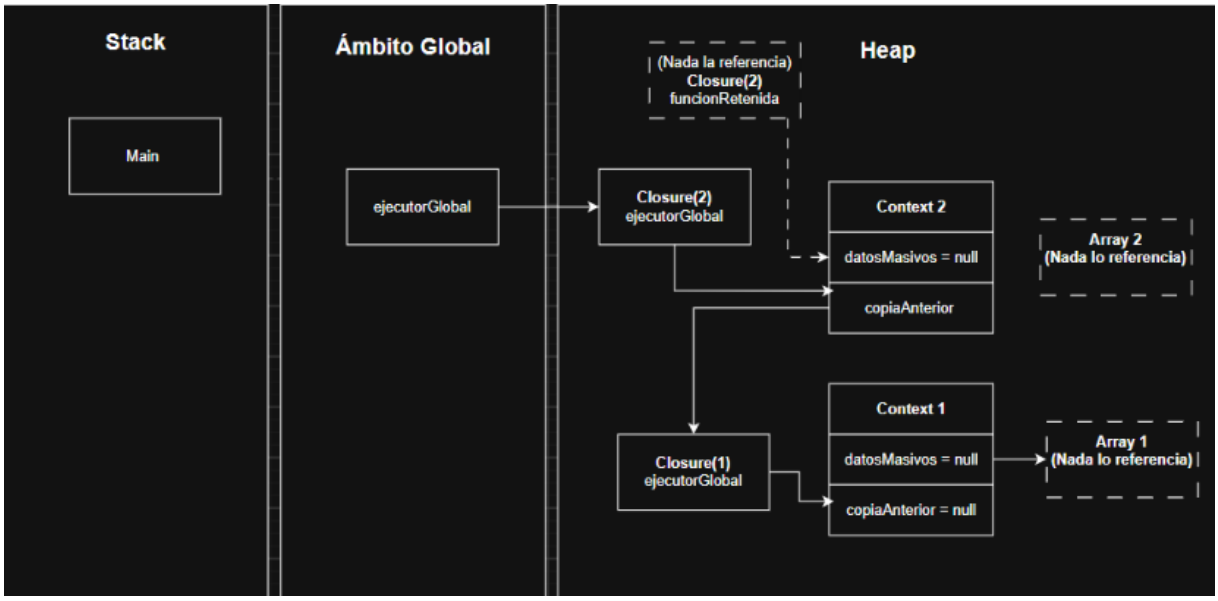


Figura 6: Fase 6

Fase 6: Para este momento si se pone, dentro del código, datosMasivos = null cuando funcionRetenida ya fue usada, Context 1 y Context 2 siguen existiendo y ejecutorGlobal sigue referenciada por la variable global, pero ahora datosMasivos vale null, por lo que ninguno de los dos Context apunta a un Array. Aquí viene lo importante, los dos Array dejaron de estar en rojo porque ya no son alcanzables desde ninguna raíz, así que el Garbage Collector puede desecharlos. Lo único que queda es una cadena mínima de Closure y Context por medio de copiaAnterior, de peso despreciable.



Caso de Estudio 2: Grafo Circular y Referencias Débiles

2.1 Código a Diagramar:

```
// Caso de Estudio 2: Grafo Circular y Referencias Débiles
class NodoGrafo {
  constructor(id) {
    this.id = id;
    this.vecinos = [];
  }

  conectarCon(nodo) {
    this.vecinos.push(nodo);
  }
}

function construirGrafoCiclico() {
  let nodoA = new NodoGrafo("A");
  let nodoB = new NodoGrafo("B");
  let nodoC = new NodoGrafo("C");

  // Conexión circular bidireccional
  nodoA.conectarCon(nodoB);
  nodoB.conectarCon(nodoC);
  nodoC.conectarCon(nodoA);

  // Registro en un gestor centralizado
  gestorCache.set(nodoA.id, nodoA);

  return nodoA.id;
}

const gestorCache = new Map(); // ¿Map o WeakMap?

let idRaiz = construirGrafoCiclico();

// El estudiante debe analizar qué sucede al ejecutar:
// gestorCache.delete(idRaiz);
```

2.2 Descripción del caso:

En este caso se tiene una clase NodoGrafo, que guarda un id y una lista de vecinos, y un método conectarCon que agrega otro nodo a esa lista. La función construirGrafoCiclico crea tres nodos, A, B y C, y los conecta en círculo: A apunta a B, B apunta a C y C vuelve a apuntar a A. Luego el nodo A se guarda en un Map llamado gestorCache usando su id como clave, y la función devuelve ese id, que queda en la variable idRaiz. Lo que se pide analizar es qué ocurre al ejecutar gestorCache.delete(idRaiz). Mientras el Map tenga guardado al nodo A, los tres nodos son alcanzables, porque desde A se llega a B y a C.

2.3 Diagrama Digitalizado:

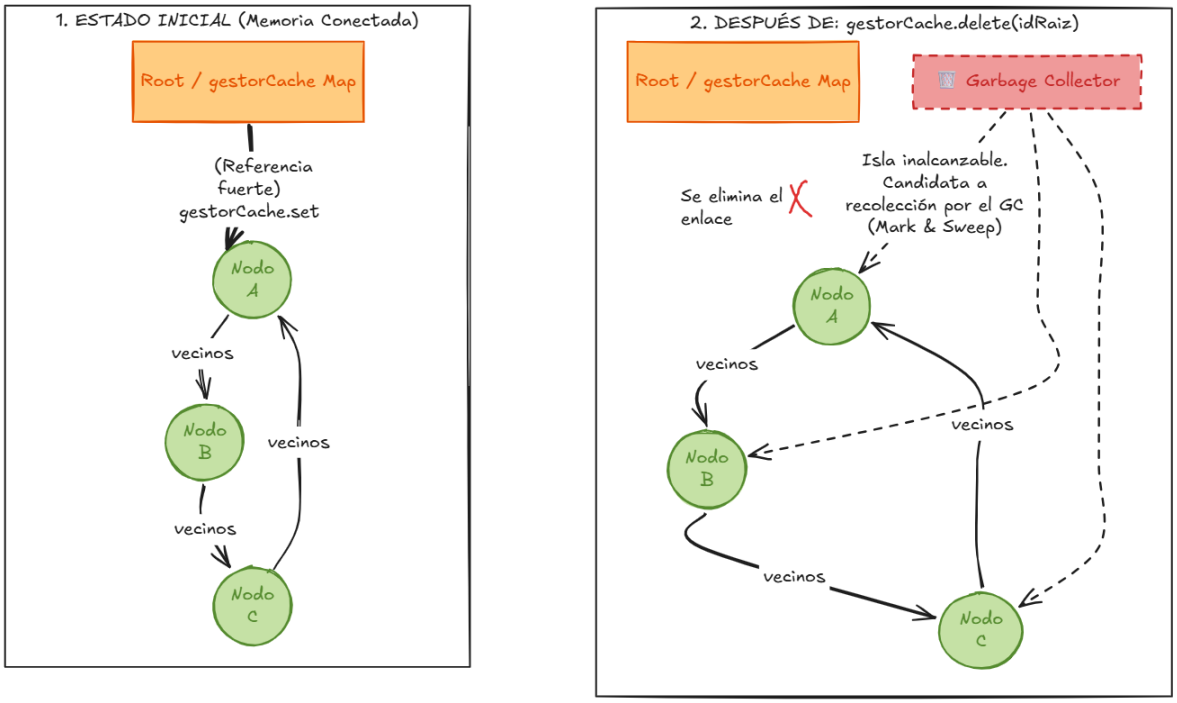


Figura 7: Diagrama digitalizado

Explicación: Este diagrama ilustra la evolución de la memoria en dos estados: inicialmente, la variable raíz que sería gestorCache (bloque naranja) mantiene una referencia fuerte hacia el Nodo A, lo que mantiene vivo a todo el grupo de nodos interconectados. Pero, en el segundo estado, al ejecutar gestorCache.delete(idRaiz) se destruye este único enlace directo con la raíz dejando a los nodos atrapados en una isla de aislamiento donde solo se apuntan entre sí, esta desconexión hacia el exterior es la que permite al Garbage Collector utilizando su algoritmo de Mark and Sweep, rastrear la memoria y detectar que el ciclo completo es inalcanzable (líneas punteadas), marcando así a los tres nodos como basura para liberar su espacio.



Caso de Estudio 3: Event Loop Queue & Retención por Callbacks

3.1 Código a Diagramar:

```
// Caso de Estudio 3: Event Loop Queue & Retención por Callbacks
const EventEmitter = require('events');
const emisorGlobal = new EventEmitter();

class GestorSesionUsuario {
  constructor(idUsuario) {
    this.idUsuario = idUsuario;
    // Reserva de 5 MB de datos por sesión
    this.bufferSesion = new Uint8Array(5 * 1024 * 1024);

    // Suscripción al emisor global
    this.oyenteEvento = () => {
      this.actualizarActividad();
    };

    emisorGlobal.on('pingSistema', this.oyenteEvento);
  }

  actualizarActividad() {
    console.log(`Sesión ${this.idUsuario} activa.`);
  }

  cerrarSesion() {
    console.log(`Cerrando sesión de ${this.idUsuario}`);
    // NOTA: El desarrollador olvidó desregistrar el evento en emisorGlobal
  }
}

function simularTrafico() {
  for (let i = 0; i < 50; i++) {
    let usuario = new GestorSesionUsuario(`USR_${i}`);
    usuario.cerrarSesion();
    // La variable local `usuario` sale de alcance (Stack) al terminar la iteración
  }
}

simularTrafico();
// Forzar la ejecución del evento global
emisorGlobal.emit('pingSistema');
```

3.2 Descripción del caso:

El programa crea un emisor global llamado emisorGlobal y una clase GestorSesionUsuario. Cada instancia reserva un Uint8Array de $5 \times 1024 \times 1024$ bytes, equivalente a 5 MiB (aproximadamente 5 MB en el uso común). Además, cada instancia crea una función oyenteEvento y la registra para el evento “pingSistema” mediante emisorGlobal.on().

La función simularTrafico() repite el proceso 50 veces, crea un usuario, llama a cerrarSesion() y deja que la variable local usuario salga de alcance. El método cerrarSesion() solo imprime un mensaje, pero no elimina el oyente del emisor. Luego, emisorGlobal.emit(’pingSistema’) ejecuta los 50 callbacks que siguen registrados.

Los buffers representan en conjunto 50×5 MiB $= 250$ MiB de datos de sesión, sin contar el resto de objetos y la sobrecarga del entorno. La cantidad exacta que se observe en memoria depende de Node.js y de su implementación; el análisis describe la retención lógica provocada

por las referencias.

3.3 Diagrama digitalizado:

1. Estado inicial: se crea y registra una sesión

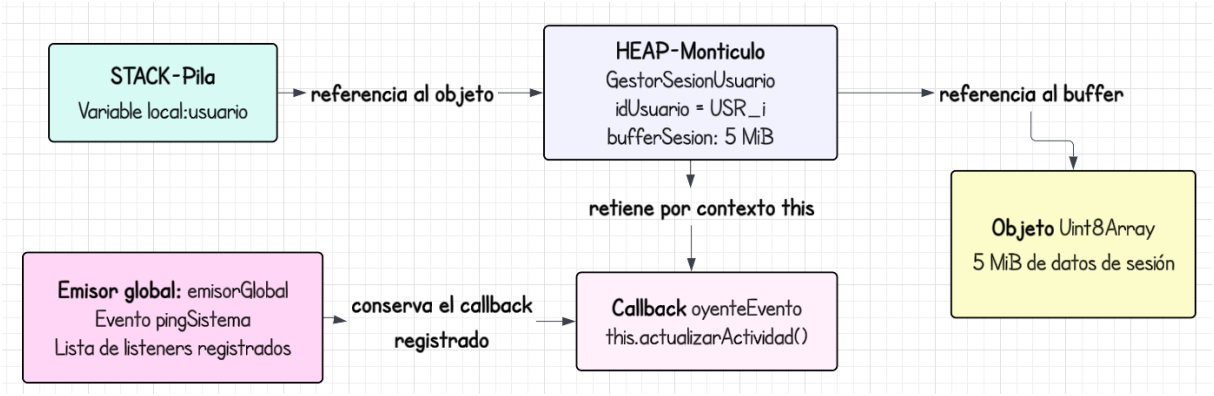


Figura 8: Estado inicial

Explicación: la variable local usuario mantiene una referencia a la instancia GestorSesionUsuario. La instancia contiene el identificador del usuario y una referencia al buffer de 5 MiB. A su vez, emisorGlobal conserva el callback oyenteEvento dentro de la lista de listeners del evento pingSistema. Como el callback utiliza this.actualizarActividad(), mantiene una referencia a la instancia correspondiente (su contexto léxico/this enlazado). Por esa ruta, la instancia y su buffer siguen alcanzables.

2. Estado después de cerrar sesión en el código original

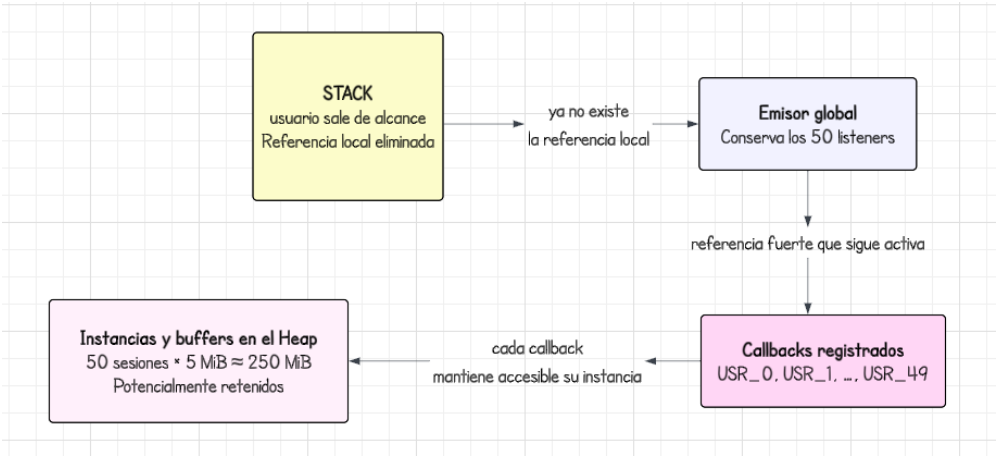


Figura 9: Estado después de cerrar sesión

Explicación: al terminar cada iteración, la referencia local usuario desaparece del Stack. No obstante, el emisor global sigue guardando el callback. El callback continúa referenciando la instancia que lo creó y esta mantiene el bufferSesion. Por tanto, que usuario salga de alcance no basta para que el Garbage Collector libere el objeto. Al finalizar las 50 iteraciones, el emisor puede retener las 50 instancias y sus buffers, lo que constituye una fuga de memoria lógica por listeners no eliminados.

Cuando se ejecuta emisorGlobal.emit('pingSistema'), cada listener registrado imprime "Sesión USR_i activa.". Este comportamiento confirma que los callbacks permanecen registrados incluso después de llamar a cerrarSesion(). No se trata de que el Event Loop cree automáticamente una referencia permanente, sino de que el emisor mantiene la lista de listeners mientras no se desregistren.

3. Estado corregido: se elimina el listener al cerrar sesión

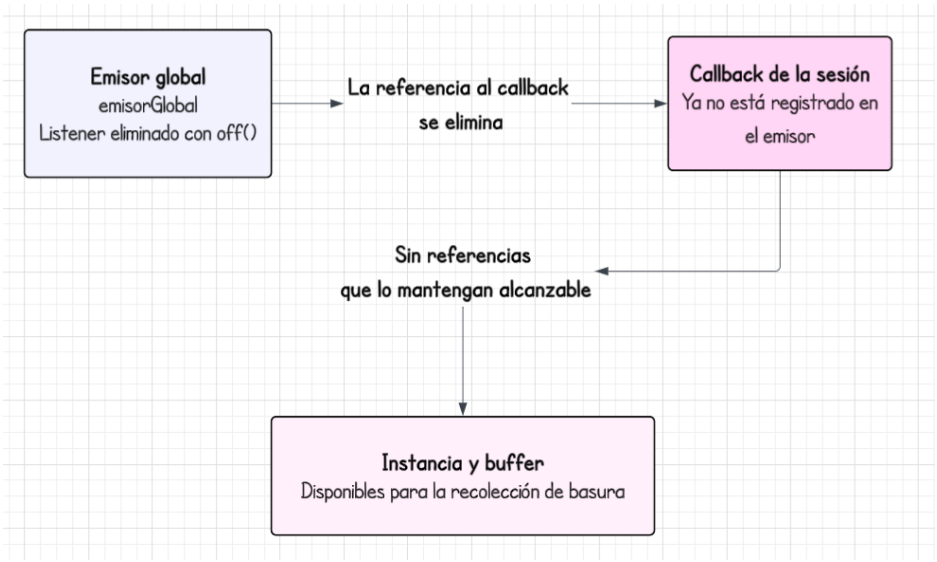


Figura 10: Estado corregido

Explicación: si cerrarSesion() elimina el callback con emisorGlobal.off('pingSistema', this.oyenteEvento), se rompe la referencia externa que retenía a la instancia. Cuando no existan otras referencias alcanzables, el Garbage Collector podrá recuperar la instancia y su buffer. La recuperación no tiene que ocurrir inmediatamente; el recolector decide cuándo ejecutarse.



Conclusiones

- En el Caso 1 se comprobó que todas las closure creadas en una misma ejecución comparten un único Context, por lo que ejecutorGlobal retiene datosMasivos aunque nunca lo use. Además, copiaAnterior encadena cada ejecución con la anterior, y el Heap crece con cada llamada hasta quedarse sin memoria.
- En el Caso 2 se concluyó que un ciclo de referencias no impide la recolección de basura, porque el Garbage Collector decide por alcanzabilidad desde las raíces y no por cuántas referencias tenga un objeto. Mientras gestorCache guardaba el nodo A, los tres nodos eran alcanzables. Al ejecutar gestorCache.delete(idRaiz), el ciclo quedó aislado y pudo ser liberado.
- En el Caso 3 se evidenció que una variable local que sale del Stack no basta para liberar un objeto, si una raíz como emisorGlobal conserva un callback que lo referencia. Los 50 listeners olvidados retuvieron las 50 instancias y sus buffers, por lo que la solución es desregistrar el listener con emisorGlobal.off() al cerrar la sesión.
- En los tres casos se confirmó que una fuga de memoria en JavaScript no es memoria olvidada, sino memoria que sigue alcanzable sin que el programa la necesite. Diagramar el Stack, el ámbito global y el Heap permite seguir la cadena de referencias desde la raíz y encontrar qué enlace mantiene vivo cada objeto.

Declaratoria de uso de IA

Se declara que, en la elaboración de este informe se utilizó la inteligencia artificial como apoyo. Su uso se limitó a explicar conceptos relacionados a los casos vistos para poder comprenderlos de mejor manera. Los códigos de los casos de estudio provienen de la guía del EVA. Los demás diagramas, hechos a mano y con herramientas digitales, y la revisión final del contenido fueron realizados por los integrantes del grupo.